

Image Basics

AddisCoder 2024 Week 2

Taught by Huy Nguyen (huynguyen@gmail.com), based on material from Alex Krentsel (alex.krentsel@gmail.com), CS106A at Stanford University, and CSC1100 at Howard University.

Warm-up with Loops, Input, and Random

Let's write a fun game together where the player guesses a random number that the computer has chosen until they get it right.

```
import random

print("Welcome to Mystery Game")

secret_number = random.randint(1,100)

guess = -1
while guess != secret_number:
    guess = int(input("Guess a number (between 1 and 100):"))
    if guess == secret_number:
        print("You solved the MYSTERY.")
    elif guess > secret_number:
        print("Lower your number! You're wrong.")
    else:
        print("Raise your number! You're wrong.")

print("The game is over.")
```

Welcome to Mystery Game

Guess a number (between 1 and 100): 61

Lower your number! You're wrong.

Guess a number (between 1 and 100): 27

Lower your number! You're wrong.

Guess a number (between 1 and 100): 14

Raise your number! You're wrong.

Guess a number (between 1 and 100): 18

Raise your number! You're wrong.

Guess a number (between 1 and 100): 20

You solved the MYSTERY.
The game is over.

Images

Let's go to the slides and talk about how images are represented.

We can look at an [online colorwheel](#) to play with the RGB concepts from lecture.

```
# We've written a simple image module for you to use. Let's import SimpleImage.

# Note: if this doesn't work, you need to make sure you've downloaded the simpleimage.py
# file, it's in the same folder as this notebook, and you've installed the Pillow library
# by running `python3 -m pip install --upgrade Pillow` in your terminal.
from simpleimage import SimpleImage

# Let's play around with it to see how it works.

# First, let's create a new image that's 10 pixels wide and 10 pixels tall.
width = 10
height = 15
canvas = SimpleImage.blank(width, height)

# Let's print out the image.
print(canvas)

<simpleimage.SimpleImage object at 0x000001C272B99F40>

# Oops, that's not very useful. Let's show the image instead.
canvas.show()
```

```
# Let's make the image bigger.
canvas.show(resize_width=200)
```

What is this thing? You learned about a lot of types, so what type is it?

```
print(type(canvas))
```

It looks like a new type? Kind of. This is a special type that we made for you to use.

We will learn more about how you can make your own types later on. For now, let's just

accept on faith that this type has been made for you to use and not worry about how it

was made.

```
<class 'simpleimage.SimpleImage'>
```

Okay, let's start painting by setting some pixels.

First, let's set the pixel at (0, 0) to be red.

```
canvas.set_rgb(0,0, 255, 0, 0)
```

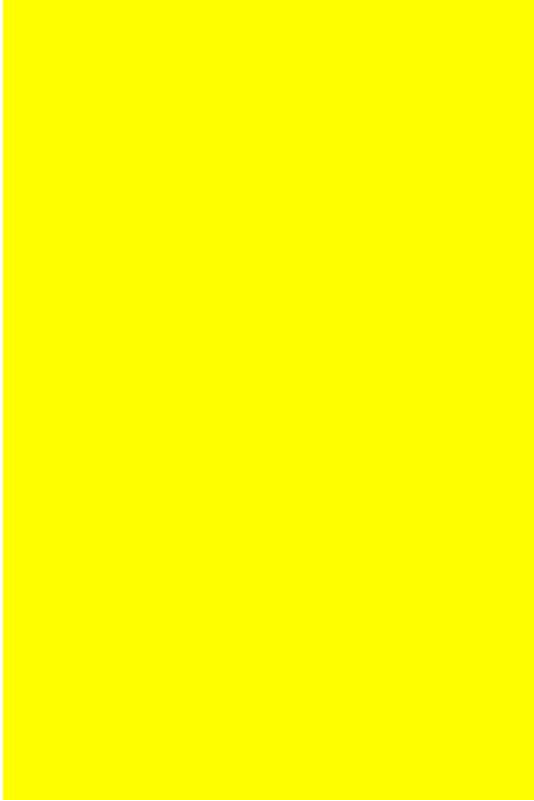
```
canvas.show(resize_width=200)
```



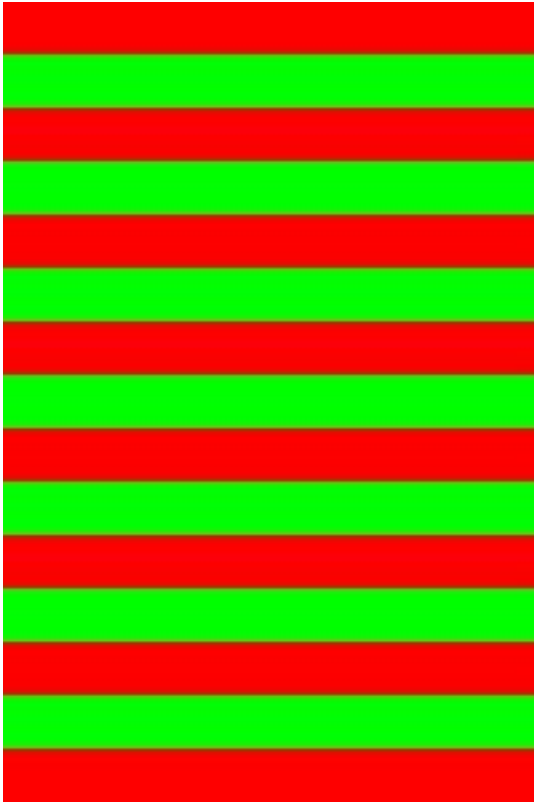
```
# We can get the width/height of the image.  
print("width:", canvas.width, "height:", canvas.height)  
width: 10 height: 15  
  
# Let's set a whole row of pixels to be red.  
for col in range(10):  
    canvas.set_rgb(col, 0, 255, 0, 0)  
  
canvas.show(resize_width=200)
```



```
# Let's set the entire image to be red.
for column in range(canvas.width):
    for row in range(canvas.height):
        canvas.set_rgb(column, row, 255, 255, 0)
canvas.show(resize_width=200)
```



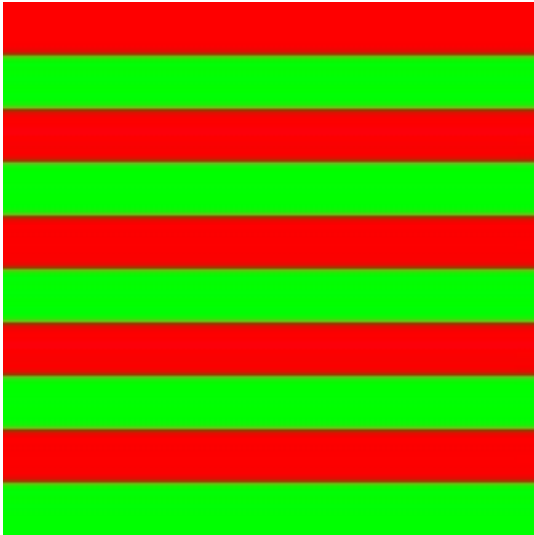
```
# How about making a striped image?
for col in range(canvas.width):
    for row in range(canvas.height):
        if row % 2 == 0:
            canvas.set_rgb(col, row, 255, 0, 0)
        else:
            canvas.set_rgb(col, row, 0, 255, 0)
canvas.show(resize_width=200)
```



Let's make this more general. Let's make a function that takes an image and make its striped.

```
def stripe_image(image):  
    for x in range(image.width):  
        for y in range(image.height):  
            if y % 2 == 0:  
                image.set_rgb(x, y, 255, 0, 0)  
            else:  
                image.set_rgb(x, y, 0, 255, 0)  
    return image
```

```
canvas = SimpleImage.blank(10, 10)  
canvas.show(resize_width=200)  
canvas = stripe_image(canvas)  
canvas.show(resize_width=200)
```



```
# We can draw manually on a canvas.
canvas = SimpleImage.blank(10,10)
canvas.show(resize_width=200)

# Make 2 eyes
print("Adding eyes")
canvas.set_rgb(3,3,0,0,0)
canvas.show(resize_width=200)
canvas.set_rgb(7,3,0,0,0)
canvas.show(resize_width=200)

# Make a nose
print("Adding a nose")
canvas.set_rgb(5,5,0,0,0)
canvas.show(resize_width=200)

# Make a mouth
```

```
print("adding mouth")
canvas.set_rgb(3,6,0,0,0)
canvas.set_rgb(4,7,0,0,0)
canvas.set_rgb(5,7,0,0,0)
canvas.set_rgb(6,7,0,0,0)
canvas.set_rgb(7,6,0,0,0)
# for x in range(3, 8):
#     canvas.set_rgb(x,7,0,0,0)
canvas.show(resize_width=200)
```

Adding eyes

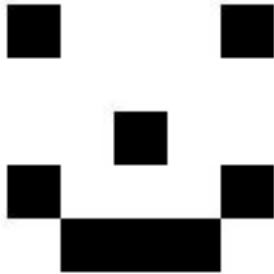




Adding a nose



adding mouth



```
# Finally, let's combine what we've learned to make a random image.
import random

random_image = SimpleImage.blank(10, 10)

for col in range(random_image.width):
    for row in range(random_image.height):
        # set the pixel at position (r, c) to be red
        randr = random.randint(0, 255)
        randg = random.randint(0, 255)
        randb = random.randint(0, 255)
        random_image.set_rgb(col, row, randr, randg, randb)

# show the image
random_image.show(resize_width=300)
```



we can get the color of a particular pixel:

```
col = 5
row = 7
pixel = random_image.get_rgb(col, row)
print(pixel)
```

```
[186, 194, 228]
```

Aside on SimpleImage

You may be wondering, what *is* SimpleImage?

For this class, you can understand SimpleImage as a custom complex type that we've created for you to use for working with canvases and images. In future classes, you'll learn more about how to create your own custom types.

As a reminder, here are the image functions we've learned this morning.

```
# Create image:
image = SimpleImage.blank(400, 200)    # create new image of size 400,
200

# Access size:
w = image.width
h = image.height
```

```
# Get pixel at x,y (x is the column number, y is the row number):
pix = image.get_rgb(x, y)
# pix is an RGB list like [100, 200, 0], where the color components of
the
# pixel are Red = 100, Green = 200, and Blue = 0.

# Set pixel at x,y:
image.set_rgb(x, y, r, g, b)

# Show image on screen:
image.show()

# Show a small image zoomed-in (larger) so it's easier to see the
pixels:
image.show(resize_width=200)
```